

# Asignatura: Fundamentos de algoritmia

## Evaluación continua. Cuestionario.

Grupo A.  
Profesor: Isabel Pita.

Nombre del alumno:

1. Tenemos una máquina capaz de realizar  $10^8$  operaciones por segundo. Tenemos una serie de algoritmos que se ejecutan en tiempo a)  $\mathcal{O}(n^2)$ , b)  $\mathcal{O}(n)$ , c)  $\mathcal{O}(n \log(n))$  y d)  $\mathcal{O}(2^n)$ . Indica que tamaño de entrada elegirías para cada uno de ellos si quieres que se ejecuten aproximadamente en un segundo: 1) 10, 2) 100, 3) 1.000, 4) 10.000, 5) 100.000, 6) 1.000.000, 7) 10.000.000, 8) 100.000.000
2. Indica la complejidad en tiempo más ajustada del siguiente algoritmo, y justifícala indicando el número de vueltas que da cada bucle y el coste de cada vuelta.

```
int n; std::cin >> n;
for (int i = 0; i < n; ++i)
    for (int j = n; j >= i; --j)
        x += v[i][j];
}
```

3. Indica la complejidad en tiempo más ajustada del siguiente algoritmo, y justifícala indicando el número de vueltas que da cada bucle y el coste de cada vuelta.

```
int n; std::cin >> n;
for (int i = 1; i < n; i = i*2) ++x;
```

4. Escribe un predicado para expresar que los valores de un vector  $v$ , entre las posiciones  $i$  y  $j$  están ordenados en orden creciente. Los valores del vector pueden ser repetidos.
5. Especifica un algoritmo que dado un vector de números enteros, cuya longitud es mayor que cero, calcule la longitud de la secuencia creciente más larga.
6. Dada la especificación:

$P:\{v.size() > 0\}$   
resolver(vector<int> v) dev int sol  
 $Q:\{sol == \#k : 0 \leq k < v.size() : v[k] > (max\ j : k < j < v.size() : v[j])\}$

Y dada la siguiente implementación:

```
int resolver(std::vector<int> const& v) {
    int i = v.size()-1; int cont = 0; int m = v[v.size()-1];
    while (i > 0) {
        if (v[i-1] > m) {
            m = v[i-1];
            ++cont;
        }
        --i;
    }
    return cont;
}
```

Indica un invariante del bucle que nos permita probar la corrección del mismo. Recuerda que en el invariante se debe expresar lo que se calcula en todas las variables que intervienen en el bucle, no solo en la variable que es devuelta como resultado de la función.

7. Indica cuál es la recurrencia que permite calcular el coste en el caso peor del siguiente algoritmo, haz el desplegado e indica el coste final del algoritmo.

```
int f ( std::vector<int> const & v, int ini, int fin){
    if (ini == fin) return 0; // Vector vacio
    else {
        int m = (ini + fin - 1) / 2;
        int x1 = f(v,ini,m+1);
        int x2 = f(v,m+1,fin);
        return x1 + x2;
    }
}
```

8. Una función recursiva que permite resolver el problema de la búsqueda binaria es  $(m = (ini + fin)/2)$  y  $fin$  el siguiente al último elemento):

$$binarySearch(v, ini, fin, x) = \begin{cases} x == v[ini] & \text{if } ini + 1 == fin \\ binarySearch(v, ini, m, x) & \text{if } ini + 1 < fin \wedge x < v[m] \\ binarySearch(v, m, fin, x) & \text{if } ini + 1 < fin \wedge v[m] \leq x \end{cases}$$

Siguiendo este ejemplo escribe una función recursiva que permita resolver el problema de calcular la suma de los valores de un vector.

9. De los algoritmos de ordenación que conoces: inserción, selección, burbuja, mergesort y quicksort, indica cual de ellos es más eficiente para ordenar un vector cuyos elementos están casi ordenados. Indica el orden de complejidad del algoritmo en el caso peor (elementos desordenados) y en el caso mejor (elementos casi ordenados).
10. De los algoritmos de ordenación que conoces: inserción, selección, burbuja, mergesort y quicksort, indica cual de ellos es más eficiente para ordenar un vector cuyos elementos son números aleatorios. Indica el orden de complejidad del algoritmo en el caso peor y en el caso mejor.